

# 25 Python Projects - DA-HYD-023

---

## Data Analysis Batch | Python Project Portfolio

**Project Scope:** These projects are designed around the concepts covered in the DA-HYD-023 Python module: fundamentals, Datatypes, functions, OOP, basic modules, exception handling, and simple TXT file handling.

### Implementation Expectation

- Menu-driven or clearly structured program flow
- Reusable functions instead of monolithic code
- Classes and objects wherever OOP naturally fits
- Input validation and exception handling
- Simple TXT-based persistence where needed
- Proper README, GitHub commits, and project explanation

# **Project 1: Student Academic Management System**

## **Problem Statement**

Educational institutions need a structured system to maintain student details, subject marks, attendance, and overall academic performance. Build a Python application that allows staff to manage student records and generate useful academic summaries without using a database.

## **Modules / Features**

- Student Registration
- Course and Subject Management
- Marks Entry
- Grade Calculation
- Attendance Tracking
- Student Search
- Topper Identification
- Low-Performance Identification
- Academic Report Generation
- Simple TXT-Based Record Storage

## **Concepts Used from the Course**

- Variables and Data Types
- Lists, Tuples, Sets, Dictionaries
- Conditional Statements and Loops
- Functions
- Classes and Objects
- Exception Handling
- Basic Modules
- TXT File Handling

## **Expected Outcome**

Students build a complete academic management application and learn how Python fundamentals, collections, functions, OOP, validations, and basic file persistence can work together in a real-world scenario.

## **Project 2: Banking Transaction Management System**

### **Problem Statement**

Banks manage multiple customers and account operations such as deposits, withdrawals, fund transfers, and balance checks. Build a Python banking application that performs these operations with proper validation and transaction logic.

### **Modules / Features**

- Customer Registration
- Account Creation
- Deposit
- Withdrawal
- Fund Transfer
- Balance Enquiry
- Minimum Balance Validation
- Transaction History
- Account Closure
- Simple Account Record Storage

### **Concepts Used from the Course**

- Functions
- Dictionaries
- Classes and Objects
- Constructors
- Encapsulation
- Inheritance
- Exception Handling
- Basic Modules
- TXT File Handling

### **Expected Outcome**

Students understand transaction processing, validation, class design, exception handling, and how to organize a practical banking workflow using Core Python.

## **Project 3: Employee Payroll Management System**

### **Problem Statement**

Organizations need to maintain employee information and calculate salaries based on role, attendance, allowances, deductions, and incentives. Build a payroll application using Python.

### **Modules / Features**

- Employee Registration
- Department Management
- Attendance Input
- Basic Salary Management
- Allowance Calculation
- Deduction Calculation
- Bonus and Incentive Calculation
- Net Salary Calculation
- Employee Search
- Payroll Summary

### **Concepts Used from the Course**

- Lists and Dictionaries
- Functions
- Classes and Objects
- Inheritance
- Method Overriding
- Exception Handling
- Basic Modules
- TXT File Handling

### **Expected Outcome**

Students learn to convert business salary rules into reusable Python functions and classes while strengthening OOP and calculation-based problem solving.

## **Project 4: Hospital Patient and Appointment Management System**

### **Problem Statement**

Hospitals need to maintain patient information, doctor details, appointments, and consultation charges. Develop a Python application that models these entities and allows basic hospital operations to be managed efficiently.

### **Modules / Features**

- Patient Registration
- Doctor Registration
- Doctor Specialization
- Appointment Booking
- Appointment Cancellation
- Doctor Allocation
- Consultation Fee Calculation
- Patient Search
- Appointment Summary
- Simple Patient Record Storage

### **Concepts Used from the Course**

- Lists
- Dictionaries
- Functions
- Classes and Objects
- Multiple Classes
- Inheritance where applicable
- Exception Handling
- Basic Modules
- TXT File Handling

### **Expected Outcome**

Students learn to design applications involving multiple interacting entities such as patients, doctors, and appointments using structured Python logic.

## **Project 5: Inventory and Stock Management System**

### **Problem Statement**

Retail stores and small warehouses need to track products, categories, stock movement, and available quantity. Build an inventory system that allows users to maintain and monitor stock using Python.

### **Modules / Features**

- Product Registration
- Product Categories
- Stock Addition
- Stock Removal
- Product Search
- Quantity Validation
- Low-Stock Identification
- Inventory Value Calculation
- Stock Summary
- Simple TXT-Based Product Storage

### **Concepts Used from the Course**

- Dictionaries
- Sets
- Functions
- Classes and Objects
- Conditional Logic
- Exception Handling
- Basic Modules
- TXT File Handling

### **Expected Outcome**

Students learn to handle inventory-style business logic, structured collections, validations, and file-based data persistence using Python.

## **Project 6: E-Commerce Shopping Cart System**

### **Problem Statement**

Online shopping applications need product browsing, cart management, discount logic, and order processing. Build the backend logic of a simple e-commerce system using Core Python.

### **Modules / Features**

- Customer Registration
- Product Catalogue
- Product Search
- Add to Cart
- Remove from Cart
- Quantity Update
- Discount Calculation
- Checkout
- Invoice Generation
- Order Summary

### **Concepts Used from the Course**

- Lists
- Dictionaries
- Sets
- Functions
- \*args and \*\*kwargs where useful
- Classes and Objects
- Exception Handling
- Basic Modules
- TXT File Handling

### **Expected Outcome**

Students understand the core programming logic behind e-commerce workflows and how collections, functions, and OOP can be combined in one project.

## **Project 7: Library Management System**

### **Problem Statement**

Libraries need to maintain books, members, issue records, returns, and fines. Build a Python library management system that supports the major day-to-day activities of a small library.

### **Modules / Features**

- Book Registration
- Book Categories
- Member Registration
- Book Search
- Book Issue
- Book Return
- Availability Tracking
- Due Date Calculation
- Fine Calculation
- Borrowing Summary

### **Concepts Used from the Course**

- Lists
- Dictionaries
- Functions
- Classes and Objects
- Date/Time Module Basics
- Exception Handling
- TXT File Handling
- Basic Modules

### **Expected Outcome**

Students build a CRUD-style Python project and learn how application state, validations, functions, and OOP work together.

## **Project 8: Hotel Reservation Management System**

### **Problem Statement**

Hotels need to manage rooms, guests, room availability, bookings, cancellations, and billing. Build a Python-based hotel reservation application using the concepts covered in class.

### **Modules / Features**

- Room Management
- Guest Registration
- Room Availability
- Room Booking
- Booking Cancellation
- Check-In
- Check-Out
- Additional Charge Calculation
- Bill Generation
- Booking Summary

### **Concepts Used from the Course**

- Classes and Objects
- Inheritance
- Lists and Dictionaries
- Functions
- Conditional Statements
- Exception Handling
- Basic Modules
- TXT File Handling

### **Expected Outcome**

Students understand booking workflows, state changes, billing calculations, and OOP-based application design through a practical reservation system.

## **Project 9: Retail Billing and Order Processing System**

### **Problem Statement**

Retail businesses need a system to manage products, quantities, discounts, taxes, and customer billing. Develop a Python application that handles the complete order-processing flow.

### **Modules / Features**

- Product Management
- Customer Management
- Product Selection
- Quantity Validation
- Stock Validation
- Discount Management
- GST Calculation
- Order Placement
- Invoice Generation
- Order Summary

### **Concepts Used from the Course**

- Lists
- Dictionaries
- Functions
- Classes and Objects
- String Formatting
- Exception Handling
- Conditional Logic
- TXT File Handling

### **Expected Outcome**

Students learn how to build transaction-based applications and explain the flow of product selection, validation, billing, and order completion in interviews.

## **Project 10: Restaurant Order Management System**

### **Problem Statement**

Restaurants require a simple system to manage menus, tables, customer orders, quantities, discounts, and final bills. Build a menu-driven restaurant application using Python.

### **Modules / Features**

- Menu Management
- Food Categories
- Table Management
- Order Placement
- Quantity Management
- Order Modification
- Discount Calculation
- GST Calculation
- Bill Generation
- Daily Order Summary

### **Concepts Used from the Course**

- Nested Dictionaries
- Lists
- Functions
- Classes and Objects
- Loops
- Exception Handling
- Basic Modules
- TXT File Handling

### **Expected Outcome**

Students gain experience working with nested data, reusable functions, classes, and business calculations in a menu-driven application.

## **Project 11: Book Reservation and Waiting List System**

### **Problem Statement**

Libraries may need to handle book reservations when all copies are already issued. Build a Python system that supports reservations, waiting lists, issue and return operations, and borrowing limits.

### **Modules / Features**

- Member Management
- Book Inventory
- Book Reservation
- Waiting List
- Issue and Return
- Borrowing Limit Validation
- Fine Calculation
- Reservation Cancellation
- Overdue Identification
- Member History Summary

### **Concepts Used from the Course**

- OOP
- Inheritance
- Dictionaries
- Lists as Queues
- Functions
- Conditional Logic
- Exception Handling
- Basic Modules
- TXT File Handling

### **Expected Outcome**

Students learn to implement queue-style reservation logic and understand how multiple entities interact in an application.

## **Project 12: Retail Inventory and Sales System**

### **Problem Statement**

Retail stores need to update stock whenever products are purchased or sold. Build a Python application that connects basic inventory management with sales processing.

### **Modules / Features**

- Product Management
- Category Management
- Stock Entry
- Sales Entry
- Automatic Quantity Update
- Low-Stock Identification
- Revenue Calculation
- Product-Wise Sales Summary
- Daily Sales Summary
- Simple TXT Storage

### **Concepts Used from the Course**

- Lists
- Dictionaries
- Functions
- Classes and Objects
- Arithmetic Logic
- Exception Handling
- Basic Modules
- TXT File Handling

### **Expected Outcome**

Students understand how multiple business modules can work together while improving their confidence in functions, collections, OOP, and file handling.

## **Project 13: Personal Finance Management System**

### **Problem Statement**

Individuals need a simple way to track income, expenses, budgets, and savings. Build a Python application that helps a user maintain and summarize personal financial activity.

### **Modules / Features**

- Income Entry
- Expense Recording
- Expense Categories
- Monthly Budget
- Budget Validation
- Transaction History
- Category-Wise Expense Summary
- Savings Calculation
- Highest Expense Identification
- Monthly Financial Summary

### **Concepts Used from the Course**

- Lists
- Dictionaries
- Functions
- Classes and Objects
- Conditional Statements
- Exception Handling
- Basic Modules
- TXT File Handling

### **Expected Outcome**

Students learn how to process and summarize financial information while applying functions, collections, calculations, OOP, and validation logic.

## **Project 14: Online Examination Management System**

### **Problem Statement**

Training institutions need an application to manage questions, conduct tests, evaluate answers, and generate results. Build a Python examination system using the concepts covered in class.

### **Modules / Features**

- Student Registration
- Question Bank
- Question Categories
- Random Question Selection
- Examination Attempt
- Answer Validation
- Negative Marking
- Score Calculation
- Result Generation
- Result Summary

### **Concepts Used from the Course**

- Lists
- Dictionaries
- Random Module
- Functions
- Classes and Objects
- Exception Handling
- Conditional Logic
- TXT File Handling

### **Expected Outcome**

Students build an examination engine and strengthen their understanding of nested collections, functions, modules, result processing, and application flow.

## **Project 15: Railway Reservation Management System**

### **Problem Statement**

Railway reservation systems manage trains, passengers, seat availability, bookings, cancellations, and waiting lists. Build a simplified Python reservation application.

### **Modules / Features**

- Train Management
- Passenger Registration
- Train Search
- Seat Availability
- Ticket Booking
- Fare Calculation
- Waiting List
- Ticket Cancellation
- Seat Reallocation
- Ticket Summary

### **Concepts Used from the Course**

- OOP
- Lists
- Dictionaries
- Functions
- Conditional Statements
- Exception Handling
- Basic Modules
- TXT File Handling

### **Expected Outcome**

Students learn reservation logic, state management, validation, and object-oriented problem solving using a familiar real-world use case.

## **Project 16: Vehicle Rental Management System**

### **Problem Statement**

Vehicle rental businesses need to manage different vehicle types, customer bookings, rental duration, charges, and returns. Build a Python vehicle rental application.

### **Modules / Features**

- Vehicle Registration
- Car and Bike Categories
- Customer Registration
- Vehicle Availability
- Booking
- Rental Duration Calculation
- Rental Price Calculation
- Vehicle Return
- Late Return Charges
- Rental Summary

### **Concepts Used from the Course**

- Classes and Objects
- Inheritance
- Polymorphism
- Method Overriding
- Functions
- Exception Handling
- Basic Modules
- TXT File Handling

### **Expected Outcome**

Students clearly demonstrate inheritance, polymorphism, and method overriding through a practical project they can explain confidently in interviews.

## **Project 17: Project and Task Management System**

### **Problem Statement**

Organizations need to manage projects, team members, tasks, deadlines, priorities, and progress. Build a Python project-management application that models these activities.

### **Modules / Features**

- Project Creation
- Employee Management
- Team Allocation
- Task Creation
- Task Assignment
- Priority Management
- Deadline Input
- Status Updates
- Project Progress Calculation
- Overdue Task Identification

### **Concepts Used from the Course**

- OOP
- Dictionaries
- Sets
- Functions
- Classes and Objects
- Exception Handling
- Basic Modules
- TXT File Handling

### **Expected Outcome**

Students understand relationships between multiple classes and learn to model a real project workflow using Python.

## **Project 18: Customer and Lead Management System**

### **Problem Statement**

Businesses need to track customers, leads, follow-ups, and lead status. Build a simple Python CRM-style application using basic file handling and OOP.

### **Modules / Features**

- Customer Registration
- Lead Registration
- Lead Status Management
- Customer Search
- Interaction Notes
- Follow-Up Entry
- Lead Conversion
- Customer Segmentation
- Follow-Up Summary
- Simple TXT-Based Records

### **Concepts Used from the Course**

- Functions
- Dictionaries
- Classes and Objects
- Conditional Logic
- Exception Handling
- Basic Modules
- TXT File Handling

### **Expected Outcome**

Students build a practical business-data application and understand customer management workflows without depending on databases or advanced file formats.

## **Project 19: Student Attendance and Performance System**

### **Problem Statement**

Training institutes need to track student attendance and assessment performance to identify strong performers and students who need support. Build a Python application that combines both areas.

### **Modules / Features**

- Student Management
- Daily Attendance
- Attendance Percentage
- Subject Marks
- Average Score
- Student Ranking
- Low-Attendance Identification
- Low-Performance Identification
- Performance Categories
- Progress Summary

### **Concepts Used from the Course**

- Nested Dictionaries
- Lists
- Functions
- Comprehensions where applicable
- Lambda Functions where useful
- Exception Handling
- Basic Modules
- TXT File Handling

### **Expected Outcome**

Students learn how Python can process structured information and generate meaningful performance insights, making the project especially relevant for data-analysis learners.

## **Project 20: Advanced Bank Management System Using OOP**

### **Problem Statement**

Develop a banking application specifically designed to demonstrate the major object-oriented programming concepts that are commonly discussed in Python interviews.

### **Modules / Features**

- Customer Class
- Account Base Class
- Savings Account
- Current Account
- Deposit
- Withdrawal
- Fund Transfer
- Interest Calculation
- Account Statement
- Custom Banking Exceptions

### **Concepts Used from the Course**

- Classes and Objects
- Constructors
- Encapsulation
- Inheritance
- Abstraction
- Polymorphism
- Method Overriding
- Custom Exceptions
- Basic Modules
- TXT File Handling

### **Expected Outcome**

Students gain a strong interview-level understanding of Python OOP by implementing the major OOP concepts in a single, easy-to-explain project.

## **Project 21: Employee Role Management System**

### **Problem Statement**

Organizations contain employees with different roles and responsibilities. Build an employee-management system that models roles such as developer, manager, and intern using OOP.

### **Modules / Features**

- Employee Registration
- Department Management
- Developer Role
- Manager Role
- Intern Role
- Salary Calculation
- Performance Rating
- Incentive Calculation
- Promotion Logic
- Employee Summary

### **Concepts Used from the Course**

- Classes and Objects
- Hierarchical Inheritance
- Method Overriding
- Polymorphism
- Abstraction
- Class Methods
- Static Methods
- Exception Handling

### **Expected Outcome**

Students demonstrate strong knowledge of object-oriented design and gain practice explaining inheritance, overriding, polymorphism, and class relationships in interviews.

## **Project 22: Courier and Delivery Management System**

### **Problem Statement**

Courier companies need to manage customers, packages, delivery agents, shipping charges, and delivery status. Build a Python application that models the complete delivery workflow.

### **Modules / Features**

- Customer Management
- Package Registration
- Shipment Creation
- Delivery Agent Management
- Agent Assignment
- Shipping Charge Calculation
- Expected Delivery Input
- Delivery Status Updates
- Delivery History Summary
- Shipment Tracking

### **Concepts Used from the Course**

- Classes and Objects
- Inheritance
- Dictionaries
- Functions
- Conditional Logic
- Exception Handling
- Basic Modules
- TXT File Handling

### **Expected Outcome**

Students learn to model workflow-driven business applications and organize customer, shipment, and delivery information using Python.

## **Project 23: Online Learning Management System**

### **Problem Statement**

Online learning platforms need to maintain students, instructors, courses, enrollments, assignments, and course completion. Build a simplified LMS using Core Python.

### **Modules / Features**

- Student Registration
- Instructor Registration
- Course Creation
- Course Enrollment
- Learning Modules
- Assignment Management
- Score Management
- Course Progress
- Completion Percentage
- Certificate Eligibility

### **Concepts Used from the Course**

- OOP
- Inheritance
- Nested Dictionaries
- Functions
- Classes and Objects
- Exception Handling
- Basic Modules
- TXT File Handling

### **Expected Outcome**

Students understand class relationships and learn how a larger application can be separated into logical modules and reusable components.

## **Project 24: User Authentication and Role Management System**

### **Problem Statement**

Applications need basic authentication and authorization before allowing users to access different features. Build a Python user-access system with simple role-based control.

### **Modules / Features**

- User Registration
- Login
- Password Validation
- Admin and User Roles
- Failed Login Tracking
- Account Locking
- Password Reset
- Role-Based Menu Access
- Activity Summary
- Simple TXT-Based User Storage

### **Concepts Used from the Course**

- Strings
- Functions
- Classes and Objects
- Encapsulation
- Exception Handling
- Basic Modules
- TXT File Handling
- Conditional Logic

### **Expected Outcome**

Students understand authentication logic, validation, role-based access, and security-oriented programming concepts using only Core Python.

## **Project 25: Placement Readiness and Student Progress Management System**

### **Problem Statement**

Training institutes need a single system to track student attendance, technical assessments, project progress, practice participation, mock interviews, and overall placement readiness. Build a Python capstone application that combines the major concepts taught in the module.

### **Modules / Features**

- Student Registration
- Batch Management
- Attendance Tracking
- Python Assessment Scores
- Manual SQL / Excel / Power BI Score Entry
- Project Completion Status
- Practice Session Tracking
- Mock Interview Status
- Performance Category
- Placement Eligibility Check
- Student Search
- Top Performer Identification
- Students Needing Improvement
- Overall Progress Percentage
- Simple TXT-Based Report Storage

### **Concepts Used from the Course**

- Variables and Operators
- Strings
- Lists
- Tuples
- Sets
- Dictionaries
- Conditional Statements

- Loops
- Functions
- \*args and \*\*kwargs
- Classes and Objects
- Encapsulation
- Inheritance
- Polymorphism
- Exception Handling
- Basic Modules
- TXT File Handling

## Expected Outcome

Students build a complete interview-ready Python capstone connected to the training and placement environment. The project demonstrates Python fundamentals, functions, collections, OOP, exception handling, modules, file handling, and application-level problem solving.

## Project Evaluation Guidelines

**Problem Understanding:** Student clearly understands the requirement, workflow, and expected result.

**Python Fundamentals:** Correct usage of variables, operators, conditions, loops, strings, and collections.

**Functions and Modular Programming:** Functions are meaningful, reusable, and logically separated.

**OOP Implementation:** Classes, objects, constructors, methods, and relationships are used correctly.

**Exception Handling and Validation:** Invalid inputs, edge cases, and incorrect operations are handled properly.

**File Handling:** Simple TXT files are used appropriately where persistent data is required.

**Project Logic and Features:** Required features work correctly and follow the intended workflow.

**Code Quality:** Readable names, clean indentation, comments where needed, and reduced duplication.

**GitHub and Documentation:** Proper repository structure, README, setup steps, and meaningful commits.

**Project Demonstration / Viva:** Students should confidently explain code flow, functions, OOP design, validations, and decisions.

## **Documentation / Presentation Requirements**

- **Problem & Objective:** Clearly explain what problem the project solves and the expected outcome.
- **Approach & Features:** Present the project workflow, major modules/features, and how the solution works.
- **Python Concepts Used:** Highlight important concepts such as functions, collections, OOP, exception handling, modules, and file handling.
- **Demo & Learning:** Demonstrate the working project and explain challenges faced, solutions implemented, and key learnings.

**“Don’t just show what you built - explain why you built it, how it works, and what you learned from it.”**